

XGBoost_Estres

May 12, 2026

1 XGBoost — Predicción de Nivel de Estrés en Adolescentes

1.0.1 Dataset: Teen Mental Health (Redes Sociales, Sueño y Bienestar)

```
[3]: # Instalar si es necesario
# pip install xgboost scikit-learn pandas numpy matplotlib seaborn openpyxl

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

import xgboost as xgb
from xgboost import XGBRegressor, plot_importance

import warnings
warnings.filterwarnings('ignore')

print(' Librerías cargadas correctamente')
print(f'XGBoost version: {xgb.__version__}')
```

Librerías cargadas correctamente

XGBoost version: 3.2.0

```
[4]: # Cargar el Excel
df = pd.read_excel('datos_estres_y_ansiedad.xlsx')

print('=' * 55)
print(' INFORMACIÓN DEL DATASET')
print('=' * 55)
print(f'Shape: {df.shape}')
print(f'\nColumnas: {list(df.columns)}')
print(f'\nValores nulos: \n{df.isnull().sum()}')
```

```
print(f'\nEstadísticas descriptivas:')
df.describe().round(2)
```

```
=====
```

INFORMACIÓN DEL DATASET

```
=====
```

Shape: (1200, 13)

Columnas: ['age', 'gender', 'daily_social_media_hours', 'platform_usage', 'sleep_hours', 'screen_time_before_sleep', 'academic_performance', 'physical_activity', 'social_interaction_level', 'anxiety_level', 'addiction_level', 'depression_label', 'stress_level']

Valores nulos:

age	0
gender	0
daily_social_media_hours	0
platform_usage	0
sleep_hours	0
screen_time_before_sleep	0
academic_performance	0
physical_activity	0
social_interaction_level	0
anxiety_level	0
addiction_level	0
depression_label	0
stress_level	0
dtype:	int64

Estadísticas descriptivas:

```
[4]:
```

	age	daily_social_media_hours	sleep_hours	\
count	1200.00	1200.00	1200.00	
mean	15.93	4.54	6.45	
std	2.02	2.03	1.44	
min	13.00	1.00	4.00	
25%	14.00	2.80	5.20	
50%	16.00	4.50	6.50	
75%	18.00	6.30	7.60	
max	19.00	8.00	9.00	

	screen_time_before_sleep	academic_performance	physical_activity	\
count	1200.00	1200.00	1200.00	
mean	1.74	2.99	1.01	
std	0.72	0.58	0.58	
min	0.50	2.00	0.00	
25%	1.10	2.50	0.50	

50%	1.80	2.99	1.00
75%	2.40	3.48	1.50
max	3.00	4.00	2.00

	anxiety_level	addiction_level	depression_label	stress_level
count	1200.00	1200.00	1200.00	1200.00
mean	5.64	5.56	0.03	5.45
std	2.86	2.83	0.16	2.90
min	1.00	1.00	0.00	1.00
25%	3.00	3.00	0.00	3.00
50%	6.00	6.00	0.00	5.00
75%	8.00	8.00	0.00	8.00
max	10.00	10.00	1.00	10.00

```
[5]: # Codificación de variables categóricas
# gender: male=1, female=0
# platform_usage: Instagram=0, TikTok=1, Both=2
# social_interaction_level: low=0, medium=1, high=2

df_encoded = df.copy()

df_encoded['gender'] = df_encoded['gender'].map({'male': 1, 'female': 0})
df_encoded['platform_usage'] = df_encoded['platform_usage'].map({'Instagram': 0,
    ↳ 'TikTok': 1, 'Both': 2})
df_encoded['social_interaction_level'] = df_encoded['social_interaction_level'].
    ↳ map({'low': 0, 'medium': 1, 'high': 2})

print(' Codificación completada')
print(f'\nDistribución del target (stress_level):')
print(df_encoded['stress_level'].value_counts().sort_index())
print(f'\nRango: {df_encoded["stress_level"].min()} - {df_encoded["stress_level"].max()}')
print(f'Media: {df_encoded["stress_level"].mean():.2f}')
```

Codificación completada

Distribución del target (stress_level):

stress_level

1	134
2	108
3	116
4	139
5	129
6	114
7	106
8	104
9	122

10 128
Name: count, dtype: int64

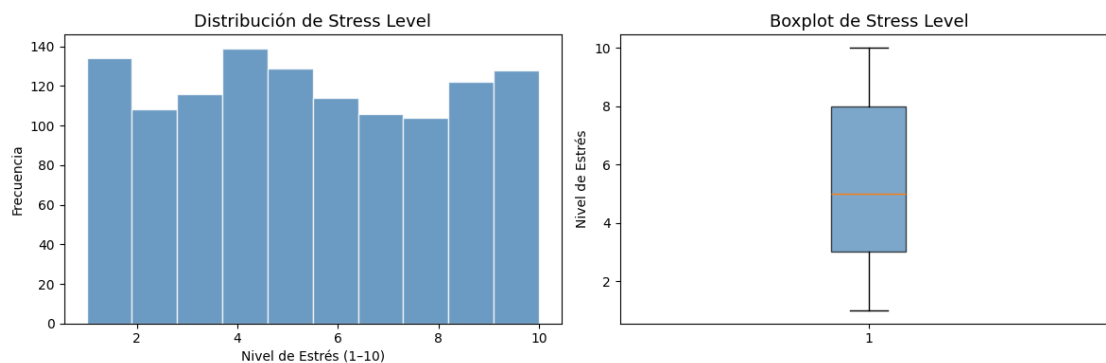
Rango: 1 - 10
Media: 5.45

```
[6]: # Visualizar distribución de la variable objetivo
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].hist(df_encoded['stress_level'], bins=10, color='steelblue',
             edgecolor='white', alpha=0.8)
axes[0].set_title('Distribución de Stress Level', fontsize=13)
axes[0].set_xlabel('Nivel de Estrés (1-10)')
axes[0].set_ylabel('Frecuencia')

axes[1].boxplot(df_encoded['stress_level'], patch_artist=True,
                boxprops=dict(facecolor='steelblue', alpha=0.7))
axes[1].set_title('Boxplot de Stress Level', fontsize=13)
axes[1].set_ylabel('Nivel de Estrés')

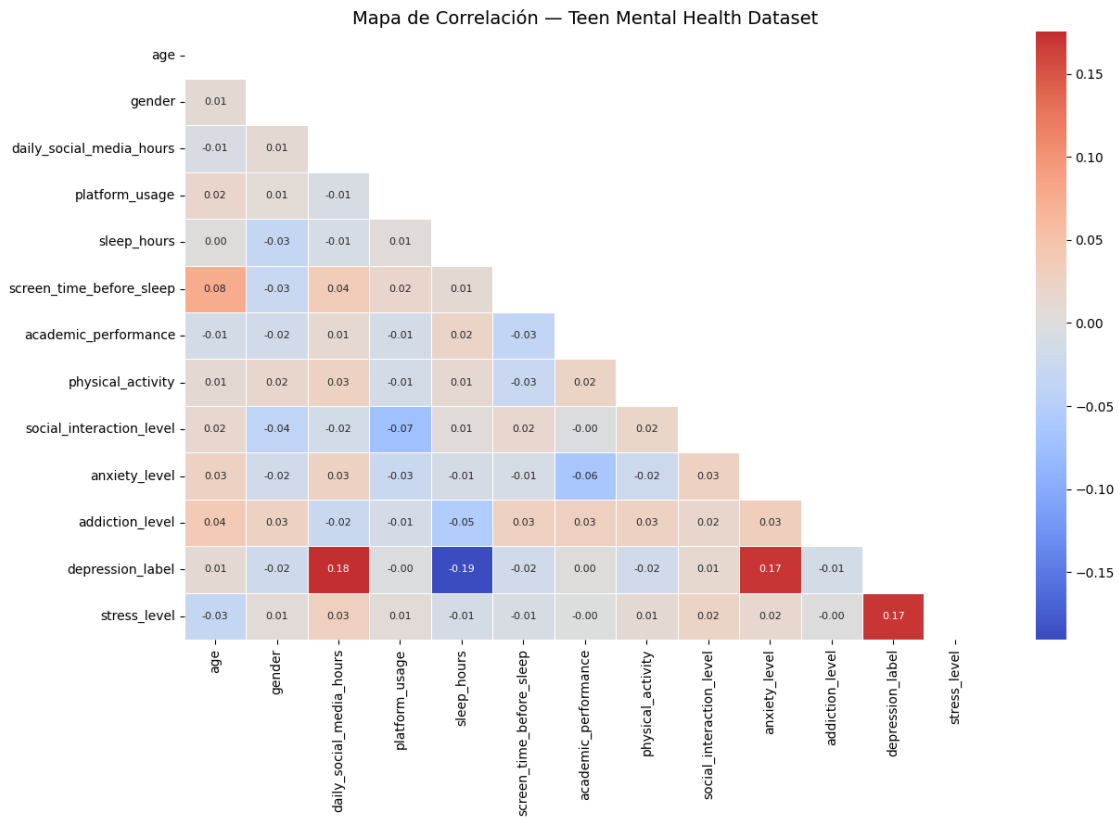
plt.tight_layout()
plt.show()
```



```
[7]: # Mapa de correlación
plt.figure(figsize=(13, 9))
corr_matrix = df_encoded.corr(numeric_only=True)
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))

sns.heatmap(
    corr_matrix, mask=mask, annot=True, fmt='.2f',
    cmap='coolwarm', center=0, linewidths=0.5,
    annot_kws={'size': 8}
)
plt.title('Mapa de Correlación - Teen Mental Health Dataset', fontsize=14)
```

```
plt.tight_layout()
plt.show()
```



```
[8]: # Separar features y target
# Excluimos depression_label (etiqueta extra) y la variable objetivo
X = df_encoded.drop(columns=['stress_level', 'depression_label'])
y = df_encoded['stress_level']

feature_names = X.columns.tolist()
print(f'Features ({len(feature_names)}): {feature_names}')
print(f'Target: stress_level | Shape: {y.shape}')

# Train / Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print(f'\n Tamaños:')
print(f' Train: {X_train.shape[0]} muestras')
```

```

print(f' Test : {X_test.shape[0]} muestras')

# Escalar (opcional para XGBoost, útil para comparaciones)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

X_train_scaled = pd.DataFrame(X_train_scaled, columns=feature_names)
X_test_scaled = pd.DataFrame(X_test_scaled, columns=feature_names)

```

Features (11): ['age', 'gender', 'daily_social_media_hours', 'platform_usage', 'sleep_hours', 'screen_time_before_sleep', 'academic_performance', 'physical_activity', 'social_interaction_level', 'anxiety_level', 'addiction_level']
Target: stress_level | Shape: (1200,)

Tamaños:
Train: 960 muestras
Test : 240 muestras

```

[9]: # Modelo con parámetros por defecto
xgb_base = XGBRegressor(
    objective = 'reg:squarederror',
    random_state = 42,
    n_jobs = -1,
    verbosity = 0
)

xgb_base.fit(X_train, y_train)
y_pred_base = xgb_base.predict(X_test)

# Métricas
def evaluar_modelo(y_true, y_pred, nombre='Modelo'):
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    # Evitar división por cero en MAPE
    mask = y_true != 0
    mape = np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) * 100

    print(f"\n{'='*45}")
    print(f' Resultados - {nombre}')
    print(f"{'='*45}")
    print(f' RMSE : {rmse:.4f}')
    print(f' MAE : {mae:.4f}')
    print(f' R² : {r2:.4f}')
    print(f' MAPE : {mape:.2f}%')

```

```

    return {'RMSE': rmse, 'MAE': mae, 'R2': r2, 'MAPE': mape}

metricas_base = evaluar_modelo(y_test, y_pred_base, 'XGBoost Base')

```

```

=====
Resultados - XGBoost Base
=====
RMSE : 3.3373
MAE  : 2.7660
R2   : -0.4364
MAPE : 94.17%

```

```

[10]: # Grid de parámetros
param_grid = {
    'n_estimators' : [100, 200, 300],
    'max_depth'    : [3, 5, 7],
    'learning_rate' : [0.05, 0.1, 0.2],
    'subsample'     : [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0],
    'min_child_weight': [1, 3]
}

xgb_cv = XGBRegressor(
    objective = 'reg:squarederror',
    random_state = 42,
    n_jobs     = -1,
    verbosity  = 0
)

grid_search = GridSearchCV(
    estimator = xgb_cv,
    param_grid = param_grid,
    cv         = 5,
    scoring    = 'neg_root_mean_squared_error',
    n_jobs     = -1,
    verbose    = 1
)

print(' Ejecutando GridSearchCV... (puede tardar unos minutos)')
grid_search.fit(X_train, y_train)

print(f'\n Mejores parámetros encontrados:')
for k, v in grid_search.best_params_.items():
    print(f'    {k:20s}: {v}')
print(f'\n    CV RMSE (mejor): {-grid_search.best_score_:.4f}')

```

Ejecutando GridSearchCV... (puede tardar unos minutos)

Fitting 5 folds for each of 216 candidates, totalling 1080 fits

Mejores parámetros encontrados:

```
colsample_bytree : 0.8
learning_rate    : 0.05
max_depth        : 3
min_child_weight : 1
n_estimators     : 100
subsample        : 0.8
```

CV RMSE (mejor): 2.9805

1.1 Modelo Optimizado

```
[11]: # Entrenar con mejores hiperparámetros
best_params = grid_search.best_params_

xgb_opt = XGBRegressor(
    **best_params,
    objective = 'reg:squarederror',
    random_state = 42,
    n_jobs     = -1,
    verbosity  = 0,
    early_stopping_rounds = 20
)

xgb_opt.fit(
    X_train, y_train,
    eval_set = [(X_train, y_train), (X_test, y_test)],
    verbose  = False
)

y_pred_opt = xgb_opt.predict(X_test)
metricas_opt = evaluar_modelo(y_test, y_pred_opt, 'XGBoost Optimizado')
```

```
=====
Resultados - XGBoost Optimizado
=====
RMSE : 2.7882
MAE  : 2.3915
R2  : -0.0026
MAPE : 83.93%
```

```
[12]: fig, axes = plt.subplots(2, 3, figsize=(18, 11))
fig.suptitle('XGBoost - Teen Mental Health: Predicción de Nivel de Estrés',
             ↪fontsize=16, fontweight='bold')
```



```

# 1. Predicciones vs Valores Reales
ax = axes[0, 0]
ax.scatter(y_test, y_pred_opt, alpha=0.6, color='steelblue',
           edgecolors='white', s=60)
lims = [min(y_test.min(), y_pred_opt.min()) - 0.5,
        max(y_test.max(), y_pred_opt.max()) + 0.5]
ax.plot(lims, lims, 'r--', linewidth=2, label='Predicción perfecta')
ax.set_xlabel('Valores Reales')
ax.set_ylabel('Predicciones')
ax.set_title('Predicciones vs Valores Reales')
ax.legend()
ax.text(0.05, 0.92, f'R² = {metricas_opt["R2"]:.4f}',
       transform=ax.transAxes, fontsize=11,
       bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

# 2. Residuos
ax = axes[0, 1]
residuos = y_test.values - y_pred_opt
ax.scatter(y_pred_opt, residuos, alpha=0.6, color='coral', edgecolors='white',
           s=60)
ax.axhline(y=0, color='black', linewidth=1.5, linestyle='--')
ax.set_xlabel('Predicciones')
ax.set_ylabel('Residuos')
ax.set_title('Gráfico de Residuos')

# 3. Distribución de Residuos
ax = axes[0, 2]
ax.hist(residuos, bins=25, color='mediumpurple', edgecolor='white', alpha=0.8)
ax.axvline(x=0, color='red', linestyle='--', linewidth=1.5)
ax.set_xlabel('Residuos')
ax.set_ylabel('Frecuencia')
ax.set_title('Distribución de Residuos')
ax.text(0.05, 0.92, f'={residuos.mean():.2f}\n={residuos.std():.2f}',
       transform=ax.transAxes, fontsize=10,
       bbox=dict(boxstyle='round', facecolor='lavender', alpha=0.7))

# 4. Feature Importance
ax = axes[1, 0]
importances = xgb_opt.feature_importances_
indices = np.argsort(importances)[::-1]
sorted_feat = [feature_names[i] for i in indices]
sorted_imp = importances[indices]

bars = ax.barh(sorted_feat[::-1], sorted_imp[::-1],
              color=plt.cm.viridis(np.linspace(0.2, 0.9, len(sorted_feat))))
ax.set_xlabel('Importancia (F-score)')

```

```

ax.set_title('Importancia de Variables')
for bar, val in zip(bars, sorted_imp[::-1]):
    ax.text(bar.get_width() + 0.001, bar.get_y() + bar.get_height()/2,
            f'{val:.3f}', va='center', fontsize=8)

# 5. Curva de Aprendizaje
ax = axes[1, 1]
results = xgb_opt.evals_result()
epochs = len(results['validation_0']['rmse'])
x_axis = range(epochs)

ax.plot(x_axis, results['validation_0']['rmse'],
        label='Train RMSE', color='steelblue', linewidth=2)
ax.plot(x_axis, results['validation_1']['rmse'],
        label='Test RMSE', color='coral', linewidth=2)
ax.set_xlabel('Número de Árboles')
ax.set_ylabel('RMSE')
ax.set_title('Curva de Aprendizaje')
ax.legend()
ax.grid(True, alpha=0.3)

# 6. Comparación Base vs Optimizado
ax = axes[1, 2]
metricas_labels = ['RMSE', 'MAE', 'R2', 'MAPE (%)']
base_vals = [metricas_base['RMSE'], metricas_base['MAE'],
             metricas_base['R2'], metricas_base['MAPE']]
opt_vals = [metricas_opt['RMSE'], metricas_opt['MAE'],
            metricas_opt['R2'], metricas_opt['MAPE']]

x_pos = np.arange(len(metricas_labels))
width = 0.35
ax.bar(x_pos - width/2, base_vals, width, label='Base',
      color='steelblue', alpha=0.8)
ax.bar(x_pos + width/2, opt_vals, width, label='Optimizado', color='coral',
      alpha=0.8)
ax.set_xticks(x_pos)
ax.set_xticklabels(metricas_labels)
ax.set_title('Comparación: Base vs Optimizado')
ax.legend()
for i, (b, o) in enumerate(zip(base_vals, opt_vals)):
    ax.text(i - width/2, b + 0.02, f'{b:.2f}', ha='center', fontsize=8)
    ax.text(i + width/2, o + 0.02, f'{o:.2f}', ha='center', fontsize=8)

plt.tight_layout()
plt.savefig('xgboost_estres_resultados.png', dpi=150, bbox_inches='tight')
plt.show()
print(" Figura guardada como 'xgboost_estres_resultados.png'")

```

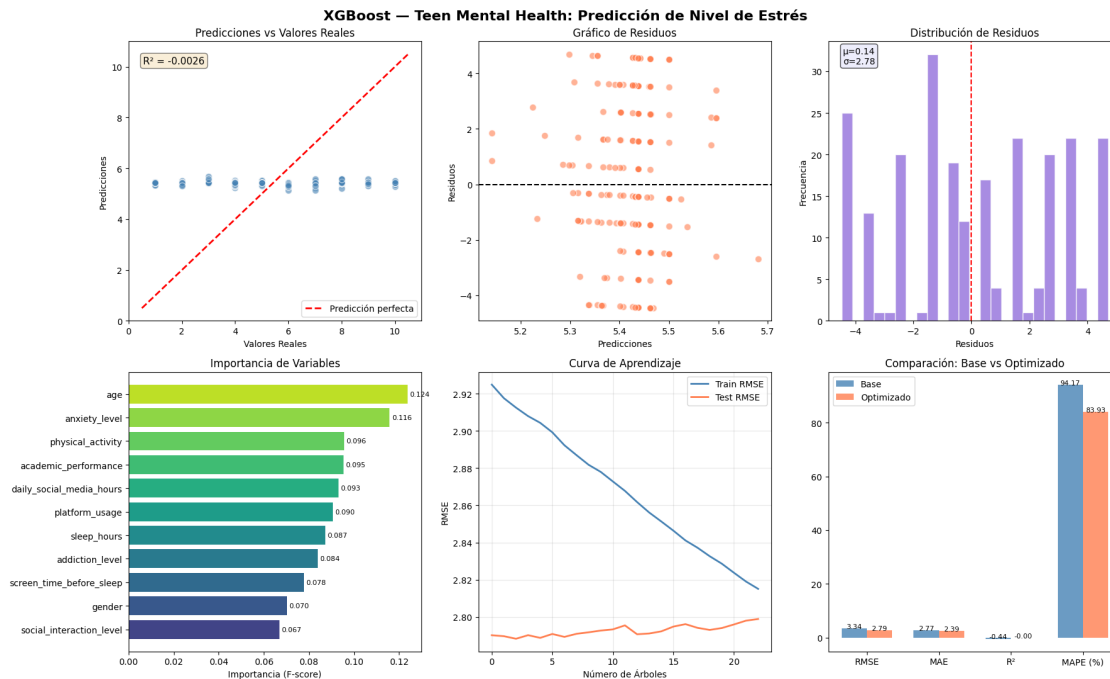


Figura guardada como 'xgboost_estres_resultados.png'

1.2 Validación Cruzada Final

```
[13]: import copy

# Cross-Validation con el modelo optimizado
xgb_opt_cv = copy.deepcopy(xgb_opt)
xgb_opt_cv.set_params(early_stopping_rounds=None)

cv_scores = cross_val_score(
    xgb_opt_cv, X, y,
    cv = 10,
    scoring = 'neg_root_mean_squared_error',
    n_jobs = -1
)

cv_rmse = -cv_scores

print('\n' + '='*50)
print('  Validación Cruzada (10-Fold)')
print('='*50)
print(f'  RMSE por fold: {np.round(cv_rmse, 3)}')
print(f'  RMSE Media   : {cv_rmse.mean():.4f}')
print(f'  RMSE Std     : {cv_rmse.std():.4f}')
```

```

print(f' IC 95%          : [{cv_rmse.mean()-2*cv_rmse.std():.4f}, '
      f'{cv_rmse.mean()+2*cv_rmse.std():.4f}']')

# Tabla resumen final
print('\n' + '='*50)
print(' RESUMEN FINAL')
print('='*50)
resumen = pd.DataFrame({
    'Métrica' : ['RMSE', 'MAE', 'R2', 'MAPE (%)'],
    'Base' : [round(metricas_base[k], 4) for k in
    ↪ ['RMSE', 'MAE', 'R2', 'MAPE']],
    'Optimizado': [round(metricas_opt[k], 4) for k in
    ↪ ['RMSE', 'MAE', 'R2', 'MAPE']]
})
resumen['Mejora (%)'] = (
    (resumen['Base'] - resumen['Optimizado']) / resumen['Base'] * 100
).round(2)
print(resumen.to_string(index=False))

```

```

=====
Validación Cruzada (10-Fold)
=====
RMSE por fold: [3.066 2.925 2.919 2.894 3.089 2.8   3.106 2.891 3.051 2.813]
RMSE Media    : 2.9554
RMSE Std      : 0.1079
IC 95%        : [2.7396, 3.1713]

```

```

=====
RESUMEN FINAL
=====

```

Métrica	Base	Optimizado	Mejora (%)
RMSE	3.3373	2.7882	16.45
MAE	2.7660	2.3915	13.54
R ²	-0.4364	-0.0026	99.40
MAPE (%)	94.1737	83.9272	10.88

1.3 Predicción

```

[14]: # Un solo registro como diccionario
# Codificación: gender (male=1, female=0) / platform (Instagram=0, TikTok=1,
    ↪ Both=2)
#          social_interaction_level (low=0, medium=1, high=2)

nuevo_registro = {
    'age' : 16,      # Edad del adolescente
    'gender' : 1,    # 1=male, 0=female

```

```

'daily_social_media_hours': 5.0,    # Horas diarias en redes sociales
'platform_usage'           : 2,      # 0=Instagram, 1=TikTok, 2=Both
'sleep_hours'              : 6.5,    # Horas de sueño
'screen_time_before_sleep' : 2.0,    # Horas de pantalla antes de dormir
'academic_performance'     : 3.0,    # Rendimiento académico (escala 0-4)
'physical_activity'        : 1.0,    # Horas de actividad física
'social_interaction_level' : 1,       # 0=low, 1=medium, 2=high
'anxiety_level'            : 5,      # Nivel de ansiedad (1-10)
'addiction_level'          : 6       # Nivel de adicción (1-10)
}

# Convertir a DataFrame
df_nuevo = pd.DataFrame([nuevo_registro])

# Predecir
prediccion = xgb_opt.predict(df_nuevo)[0]

print('=' * 45)
print('    PREDICCIÓN - Registro Individual')
print('=' * 45)
print(f'\n Nivel de estrés estimado: {prediccion:.2f} / 10')
nivel = 'Bajo' if prediccion <= 3 else ('Medio' if prediccion <= 6 else 'Alto')
print(f' Categoría                : {nivel}')

```

```

=====
    PREDICCIÓN - Registro Individual
=====

```

```

Nivel de estrés estimado: 5.44 / 10
Categoría                : Medio

```

1.4 Predicción con Múltiples Registros

```

[16]: #
# PREDICCIÓN - Múltiples Perfiles de Adolescentes
#

# Varios perfiles de adolescentes
nuevos_registros = pd.DataFrame({
    'age'           : [14, 17, 15, 18, 16 ],
    'gender'        : [0, 1, 0, 1, 0 ],
    'daily_social_media_hours': [1.5, 7.5, 4.0, 6.0, 2.5 ],
    'platform_usage' : [0, 1, 2, 1, 0 ],
    'sleep_hours'    : [8.5, 5.0, 6.5, 4.5, 7.5 ],
    'screen_time_before_sleep': [0.5, 3.0, 2.0, 2.8, 1.0 ],
    'academic_performance' : [3.8, 2.1, 3.0, 2.5, 3.5 ],
    'physical_activity' : [2.0, 0.3, 1.0, 0.5, 1.8 ],

```

```

        'social_interaction_level': [2,    0,    1,    0,    2    ],
        'anxiety_level'           : [2,    8,    5,    9,    3    ],
        'addiction_level'         : [1,    9,    5,    8,    2    ]
    })

# Etiquetas descriptivas
etiquetas = [
    'Adolescente saludable',
    'Alto uso redes + poco sueño',
    'Perfil promedio',
    'Estrés elevado + adicción',
    'Activo y equilibrado'
]

# Predecir todos los perfiles
predicciones = xgb_opt.predict(nuevos_registros)

# Crear DataFrame de resultados
resultados_df = nuevos_registros.copy()

# Insertar etiquetas
resultados_df.insert(0, 'Perfil', etiquetas)

# Predicción numérica
resultados_df['Estres_Pred'] = predicciones.round(2)

# Clasificación del nivel de estrés
resultados_df['Nivel_Estres'] = [
    'Bajo' if p <= 3 else
    ('Medio' if p <= 6 else 'Alto')
    for p in predicciones
]

# Mostrar resultados
print("\n" + "=" * 70)
print("    PREDICCIONES - Múltiples Perfiles de Adolescentes")
print("=" * 70)

cols_resumen = [
    'Perfil',
    'daily_social_media_hours',
    'sleep_hours',
    'academic_performance',
    'physical_activity',
    'anxiety_level',
    'addiction_level',
    'Estres_Pred',

```

```

    'Nivel_Estres'
]

print(resultados_df[cols_resumen].to_string(index=False))

```

```

=====
PREDICCIONES - Múltiples Perfiles de Adolescentes
=====

```

	Perfil	daily_social_media_hours	sleep_hours	academic_performance	physical_activity	anxiety_level	addiction_level	Estres_Pred	Nivel_Estres
	Adolescente saludable		1.5	8.5					
3.8	2.0	2	1	5.50				Medio	
	Alto uso redes + poco sueño		7.5	5.0					
2.1	0.3	8	9	5.40				Medio	
	Perfil promedio		4.0	6.5					
3.0	1.0	5	5	5.44				Medio	
	Estrés elevado + adicción		6.0	4.5					
2.5	0.5	9	8	5.34				Medio	
	Activo y equilibrado		2.5	7.5					
3.5	1.8	3	2	5.44				Medio	

```

[17]: #
# FUNCIÓN DE VISUALIZACIÓN
#

def visualizar_prediccion_estres(estres_pred, datos_input):

    fig, axes = plt.subplots(1, 3, figsize=(17, 5))
    fig.suptitle("Análisis de la Predicción de Estrés",
                 fontsize=15,
                 fontweight='bold')

    # 1. Distribución histórica
    ax = axes[0]

    ax.hist(y, bins=25,
            alpha=0.7,
            edgecolor='white')

    ax.axvline(estres_pred,
               color='red',
               linewidth=2.5,
               linestyle='--',
               label=f'Predicción: {estres_pred:.2f}')

```

```

ax.axvline(y.mean(),
           color='orange',
           linewidth=2,
           linestyle=':',
           label=f'Promedio: {y.mean():.2f}')

ax.set_xlabel('Nivel de Estrés')
ax.set_ylabel('Frecuencia')
ax.set_title('Distribución Histórica')
ax.legend(fontsize=8)

# 2. Percentil
ax = axes[1]

percentil = (y < estres_pred).mean() * 100

colores = [
    '#2ecc71' if i < percentil else '#ecf0f1'
    for i in range(100)
]

ax.bar(range(100), [1]*100,
       color=colores,
       width=1.0)

ax.set_xlim(0, 100)
ax.set_title(
    f'Percentil del Estrés\n'
    f'Supera al {percentil:.1f}% del dataset'
)

ax.set_xlabel('Percentil')
ax.set_yticks([])

ax.text(percentil/2,
        0.5,
        f'{percentil:.0f}%',
        ha='center',
        va='center',
        fontsize=15,
        fontweight='bold',
        color='white')

# 3. Variables clave vs promedio
ax = axes[2]

vars_radar = [

```



```

        'daily_social_media_hours',
        'sleep_hours',
        'academic_performance',
        'anxiety_level',
        'addiction_level'
    ]

    vals_input = [datos_input[v].values[0] for v in vars_radar]
    vals_media = [X[v].mean() for v in vars_radar]

    # Normalización 0-1
    vals_norm_input = [
        (v - X[c].min()) / (X[c].max() - X[c].min())
        for v, c in zip(vals_input, vars_radar)
    ]

    vals_norm_media = [
        (v - X[c].min()) / (X[c].max() - X[c].min())
        for v, c in zip(vals_media, vars_radar)
    ]

    x_pos = np.arange(len(vars_radar))
    width = 0.35

    ax.bar(x_pos - width/2,
           vals_norm_input,
           width,
           label='Perfil actual',
           alpha=0.8)

    ax.bar(x_pos + width/2,
           vals_norm_media,
           width,
           label='Promedio',
           alpha=0.8)

    ax.set_xticks(x_pos)
    ax.set_xticklabels(vars_radar,
                       rotation=15,
                       fontsize=9)

    ax.set_ylabel('Valor normalizado')
    ax.set_title('Variables Clave vs Promedio')
    ax.legend(fontsize=8)

    plt.tight_layout()
    plt.savefig('prediccion_estres_adolescente.png',

```

```

        dpi=150,
        bbox_inches='tight')

plt.show()

print("\n Gráfica guardada como "
      "'prediccion_estres_adolescente.png'")

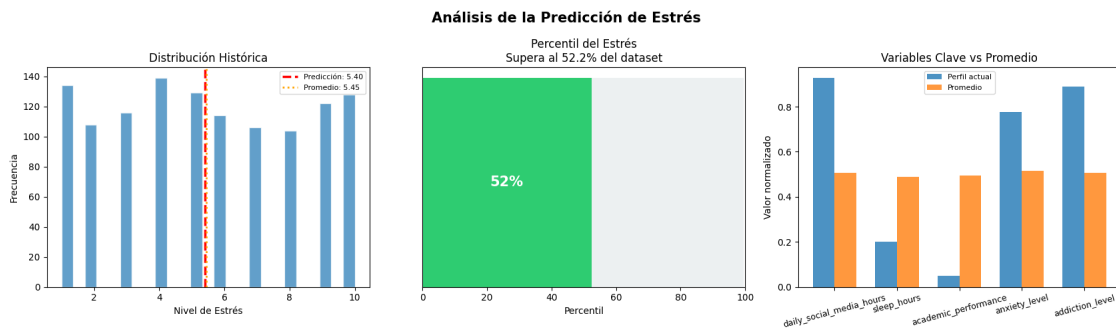
#
#  VISUALIZAR UN PERFIL ESPECÍFICO
#

indice = 1  # Cambiar perfil aquí

prediccion = predicciones[indice]
perfil_df = nuevos_registros.iloc[[indice]]

visualizar_prediccion_estres(prediccion, perfil_df)

```



Gráfica guardada como 'prediccion_estres_adolescente.png'